

Exercise 2.5 Report: Nonstationary 10-Armed Bandit

1 Question

Exercise 2.5 (programming). Design and conduct an experiment to demonstrate the difficulties that sample-average methods have for nonstationary problems. Use a modified version of the 10-armed testbed in which all the $q_*(a)$ start out equal and then take independent random walks, for example by adding a normally distributed increment with mean zero and standard deviation 0.01 to all the $q_*(a)$ on each step. Prepare plots like Figure 2.2 for an action-value method using sample averages, incrementally computed, and another action-value method using a constant step-size parameter, $\alpha = 0.1$. Use $\epsilon = 0.1$ and longer runs, for example 10,000 steps.

2 Purpose of the Experiment

The purpose of this experiment is to show why sample-average action-value estimates are not well suited to nonstationary bandit problems. In a stationary problem, the true value $q_*(a)$ of each action is fixed, so averaging all past rewards is statistically reasonable. In a nonstationary problem, however, $q_*(a)$ changes over time. Old rewards become less relevant, so an update rule that gives too much weight to old data adapts slowly.

We compare two ϵ -greedy action-value methods:

1. A sample-average method with an effective step size $\alpha_t(a) = 1/N_t(a)$.
2. A constant step-size method with $\alpha = 0.1$.

Both methods use $\epsilon = 0.1$, so with probability 0.1 the agent explores by selecting a random action, and with probability 0.9 it exploits the greedy action according to its current value estimates.

3 Experimental Setup

The experiment uses a nonstationary version of the 10-armed testbed:

- Number of actions: $k = 10$.
- Number of independent runs: 2000.
- Number of steps per run: 10,000.
- Exploration rate: $\epsilon = 0.1$.
- Reward noise standard deviation: 1.
- Random-walk standard deviation for each true action value: 0.01.

At the start of each run, all true action values are initialized equally:

$$q_*(a) = 0, \quad a = 1, 2, \dots, 10. \quad (1)$$

At every time step, after the reward is generated and the estimate is updated, every true action value independently takes a random-walk step:

$$q_*(a) \leftarrow q_*(a) + \eta_t(a), \quad \eta_t(a) \sim \mathcal{N}(0, 0.01^2). \quad (2)$$

The reward for selecting action A_t is sampled as

$$R_t \sim \mathcal{N}(q_*(A_t), 1). \quad (3)$$

4 Methods Compared

4.1 Sample-Average Method

The sample-average method updates only the selected action A_t . If $N_t(A_t)$ is the number of times action A_t has been selected up to time t , then the incremental sample-average update is

$$Q_{t+1}(A_t) = Q_t(A_t) + \frac{1}{N_t(A_t)} [R_t - Q_t(A_t)]. \quad (4)$$

This is equivalent to averaging all previously observed rewards for that action. The difficulty is that as $N_t(A_t)$ grows, the step size $1/N_t(A_t)$ becomes very small. Therefore, new rewards have less and less influence on the estimate, even though the true action value is drifting.

4.2 Constant Step-Size Method

The constant step-size method also updates only the selected action, but it uses a fixed step size:

$$Q_{t+1}(A_t) = Q_t(A_t) + \alpha [R_t - Q_t(A_t)], \quad \alpha = 0.1. \quad (5)$$

Because α does not decay over time, recent rewards continue to affect the estimate. This makes the method better able to track nonstationary action values.

5 Results

Figure 1 shows the average reward over time. The constant step-size method achieves higher reward than the sample-average method after the early part of learning. This indicates that it tracks the drifting action values more effectively.

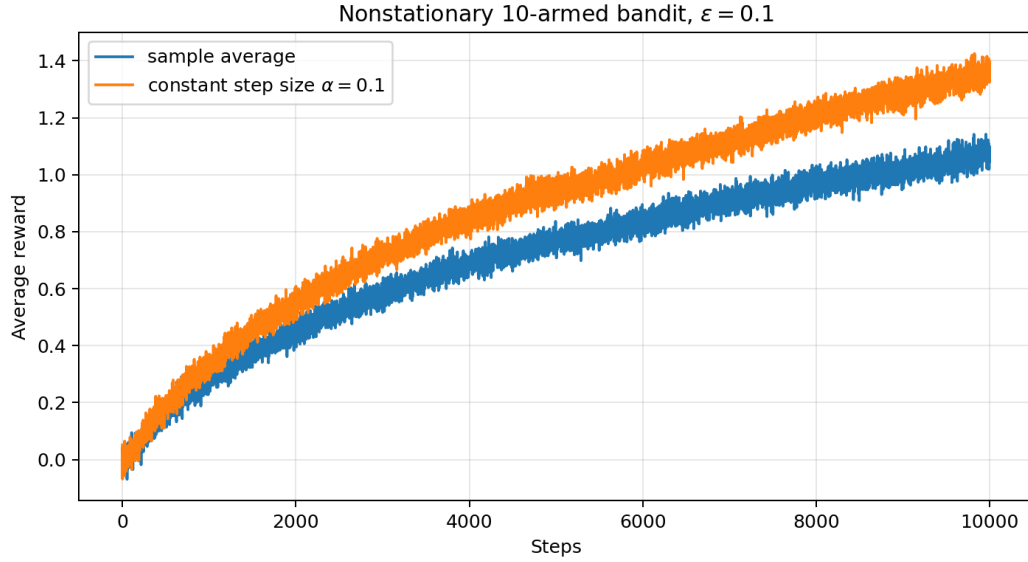


Figure 1: Average reward for the sample-average method and the constant step-size method.

Figure 2 shows the percentage of times the optimal action was selected. The constant step-size method reaches a much higher optimal-action percentage because it adapts more quickly when the identity of the best action changes.

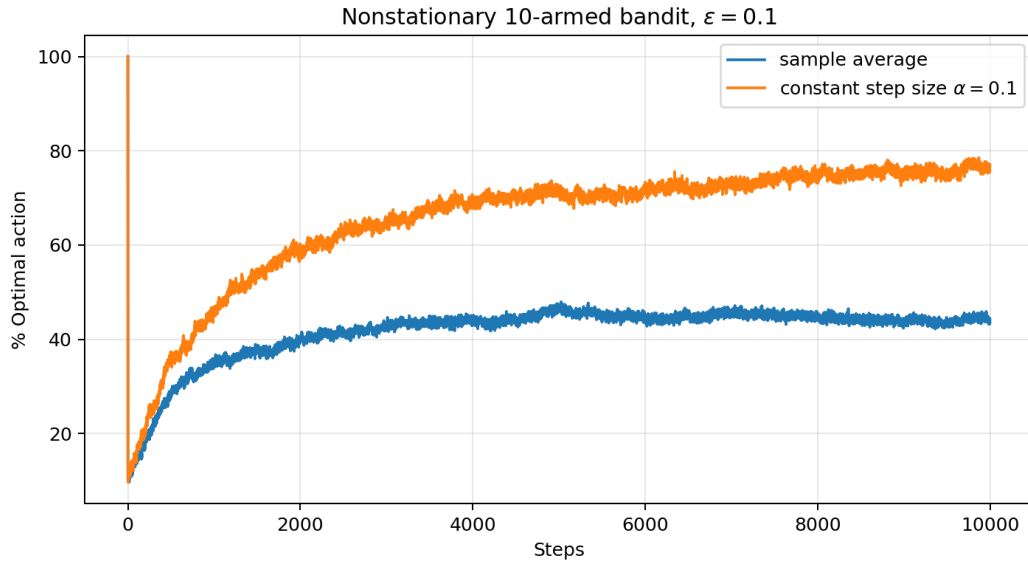


Figure 2: Percentage of optimal action selections for the sample-average method and the constant step-size method.

The final 1000-step averages from the simulation are summarized below.

Method	Final 1000-step average reward	Final 1000-step optimal action rate
Sample average	1.044	44.0%
Constant step size, $\alpha = 0.1$	1.323	75.9%

Table 1: Final performance comparison over the last 1000 steps.

6 Interpretation

The result demonstrates the main weakness of sample-average methods in nonstationary environments. In the sample-average method, the update weight for a frequently selected action is

$$\alpha_t(a) = \frac{1}{N_t(a)}. \quad (6)$$

As $N_t(a)$ increases, $\alpha_t(a)$ approaches zero. Therefore, the estimate $Q_t(a)$ becomes increasingly insensitive to new rewards. This is problematic when $q_*(a)$ is moving due to a random walk.

By contrast, the constant step-size method uses a fixed $\alpha = 0.1$. This means the value estimate always remains responsive to recent rewards. It effectively gives more weight to recent observations and discounts older observations. This behavior is desirable in a nonstationary task.

7 Conclusion

This experiment confirms that sample-average action-value methods are appropriate for stationary bandit problems but perform poorly in nonstationary settings. When the true action values drift over time, old rewards can become misleading. The sample-average method adapts too slowly because its step size decreases with the number of times an action has been selected.

The constant step-size method performs better because it maintains a fixed learning rate and continues to track recent changes in the reward distribution. For nonstationary problems, constant step-size updates are usually preferable to sample averages.

A Python Code

The following Python code generated the experiment and the two plots.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Exercise 2.5: Nonstationary 10-armed bandit
5 # Compare sample-average action-value updates vs constant step-size updates.
6
7 rng = np.random.default_rng(0)
8
9 k = 10
10 runs = 2000
11 steps = 10_000
12 epsilon = 0.1
13 alpha_const = 0.1
14 random_walk_std = 0.01
15 reward_std = 1.0
16
17 # True action values start equal at 0 for every action in every run.
18 q_true = np.zeros((runs, k))
19
```

```

20 # Estimated action values and counts for the two methods.
21 Q_sample = np.zeros((runs, k))
22 N_sample = np.zeros((runs, k), dtype=np.int32)
23
24 Q_const = np.zeros((runs, k))
25
26 avg_reward_sample = np.zeros(steps)
27 avg_reward_const = np.zeros(steps)
28 optimal_sample = np.zeros(steps)
29 optimal_const = np.zeros(steps)
30
31 run_idx = np.arange(runs)
32
33 for t in range(steps):
34     # Compute the current best true value.
35     # A selected action is counted as optimal if it has this maximum value.
36     max_q = np.max(q_true, axis=1)
37
38     # ----- Method 1: sample-average update, alpha = 1 / N_t(a) -----
39     greedy_sample = np.argmax(Q_sample, axis=1)
40     random_sample = rng.integers(k, size=runs)
41     explore_sample = rng.random(runs) < epsilon
42     A_sample = np.where(explore_sample, random_sample, greedy_sample)
43
44     R_sample = q_true[run_idx, A_sample] + rng.normal(0.0, reward_std, size=runs)
45
46     N_sample[run_idx, A_sample] += 1
47     step_size_sample = 1.0 / N_sample[run_idx, A_sample]
48     Q_sample[run_idx, A_sample] += step_size_sample * (
49         R_sample - Q_sample[run_idx, A_sample]
50     )
51
52     avg_reward_sample[t] = R_sample.mean()
53     optimal_sample[t] = np.mean(q_true[run_idx, A_sample] == max_q) * 100
54
55     # ----- Method 2: constant step-size update, alpha = 0.1 -----
56     greedy_const = np.argmax(Q_const, axis=1)
57     random_const = rng.integers(k, size=runs)
58     explore_const = rng.random(runs) < epsilon
59     A_const = np.where(explore_const, random_const, greedy_const)
60
61     R_const = q_true[run_idx, A_const] + rng.normal(0.0, reward_std, size=runs)
62
63     Q_const[run_idx, A_const] += alpha_const * (
64         R_const - Q_const[run_idx, A_const]
65     )
66
67     avg_reward_const[t] = R_const.mean()
68     optimal_const[t] = np.mean(q_true[run_idx, A_const] == max_q) * 100
69
70     # Nonstationarity: every true action value takes an independent random-walk step.
71     q_true += rng.normal(0.0, random_walk_std, size=(runs, k))
72
73 # Plot average reward.
74 fig1, ax1 = plt.subplots(figsize=(8, 4.5))
75 ax1.plot(avg_reward_sample, label="sample average")
76 ax1.plot(avg_reward_const, label=r"constant step size $\alpha=0.1$")
77 ax1.set_xlabel("Steps")
78 ax1.set_ylabel("Average reward")
79 ax1.set_title(r"Nonstationary 10-armed bandit, $\epsilon=0.1$")
80 ax1.legend()
81 ax1.grid(True, alpha=0.3)
82 fig1.tight_layout()
83 fig1.savefig("exercise_2_5_average_reward.png", dpi=180)
84
85 # Plot percentage of optimal action.
86 fig2, ax2 = plt.subplots(figsize=(8, 4.5))
87 ax2.plot(optimal_sample, label="sample average")

```

```

88 ax2.plot(optimal_const, label=r"constant step size  $\alpha=0.1$ ")
89 ax2.set_xlabel("Steps")
90 ax2.set_ylabel("% Optimal action")
91 ax2.set_title(r"Nonstationary 10-armed bandit,  $\epsilon=0.1$ ")
92 ax2.legend()
93 ax2.grid(True, alpha=0.3)
94 fig2.tight_layout()
95 fig2.savefig("exercise_2_5_percent_optimal.png", dpi=180)
96
97 print("Final 1000-step average reward:")
98 print(f"    sample average: {avg_reward_sample[-1000:].mean():.3f}")
99 print(f"    constant alpha: {avg_reward_const[-1000:].mean():.3f}")
100
101 print("Final 1000-step % optimal action:")
102 print(f"    sample average: {optimal_sample[-1000:].mean():.1f}%")
103 print(f"    constant alpha: {optimal_const[-1000:].mean():.1f}%")

```